



**SCHOOL OF MATHEMATICS AND APPLIED SCIENCES
DEPARTMENT OF COMPUTING**

TO : MR. S. YUTE

FROM : TRINITY KAMBANJE & FRANCIS NYANGU

REGISTRATION NO : BSC-COM-NE-25-23 & BSC-COM-NE-13-21

COURSE NAME : NETWORK PROGRAMMING AND APPLICATION TO
DEVELOPMENT

COURSE CODE : NET322

ASSIGNMENT TITLE : PLANNING & ARCHITECTURE

DUE DATE : 9TH MAY, 2026

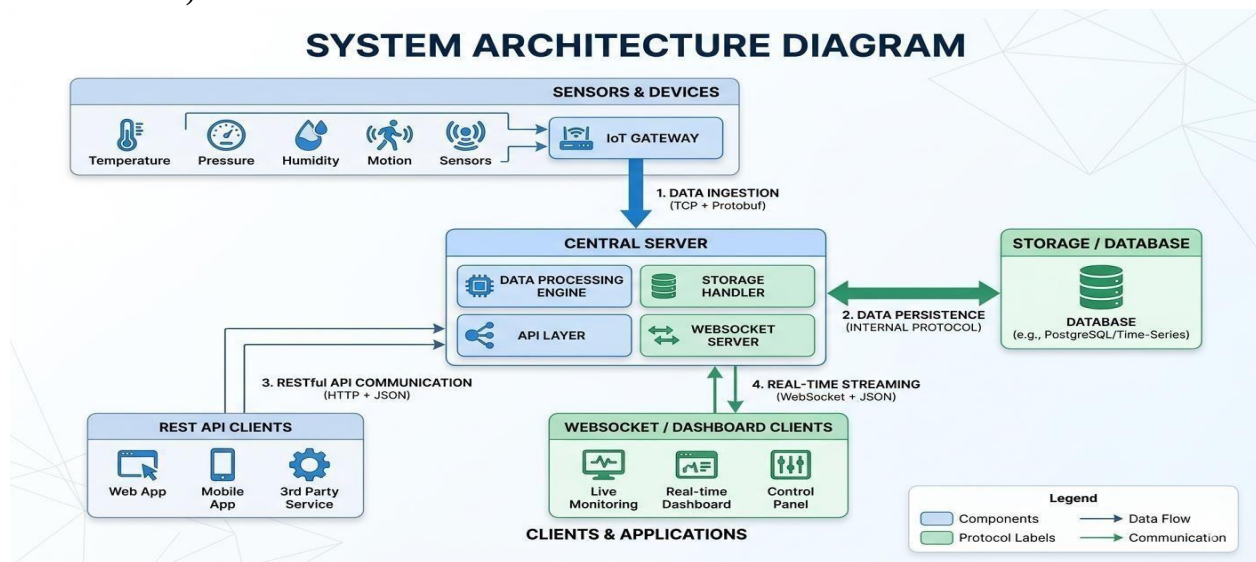
(A) SYSTEM ARCHITECTURE

The system follows a decoupled asynchronous architecture where multiple sensor devices publish readings via HTTP REST to a central AsyncIO server (FastAPI/Starlette), which then distributes data to both a PostgreSQL database for persistence and to live dashboard clients over WebSocket connections.

The architecture comprises four main component groups: sensor devices (Python scripts), the async HTTP/WebSocket server, a PostgreSQL storage layer, and two types of clients (REST clients for historical queries and WebSocket clients for live dashboards).

Each communication link uses a specific protocol and serialization format: sensors communicate with the server using HTTP/1.1 with JSON serialization, REST clients also use HTTP/1.1 with JSON, WebSocket clients use the WebSocket protocol (RFC 6455) with JSON framing, and the server connects to PostgreSQL using the binary PostgreSQL wire protocol via the asyncpg driver with prepared statements for efficiency.

Figure 1: System Architecture Diagram (sensors, server, storage, REST clients, WebSocket clients with protocols labeled)



(B). SEQUENCE DIAGRAM

The complete flow from a sensor measurement to a live dashboard update follows a seven-step asynchronous pipeline. The process begins when a sensor device performs an HTTP POST request containing its sensor ID and measured value.

The server validates the request, enriches it with a server-side timestamp, and immediately returns HTTP 202 Accepted to the sensor, ensuring the sensor is not blocked by storage latency.

Simultaneously, the server places the validated reading into an asynchronous ingress queue. A separate storage writer coroutine pulls readings from this queue and performs a batched INSERT operation into PostgreSQL.

After successful persistence, the server broadcasts the new reading to all connected WebSocket dashboard clients that have subscribed to that sensor's data stream.

The diagram explicitly includes the WebSocket upgrade handshake, which occurs when a dashboard client first connects: the client sends an HTTP GET request with Upgrade: websocket headers, the server

responds with HTTP 101 Switching Protocols, and the connection remains open for bidirectional live messaging.

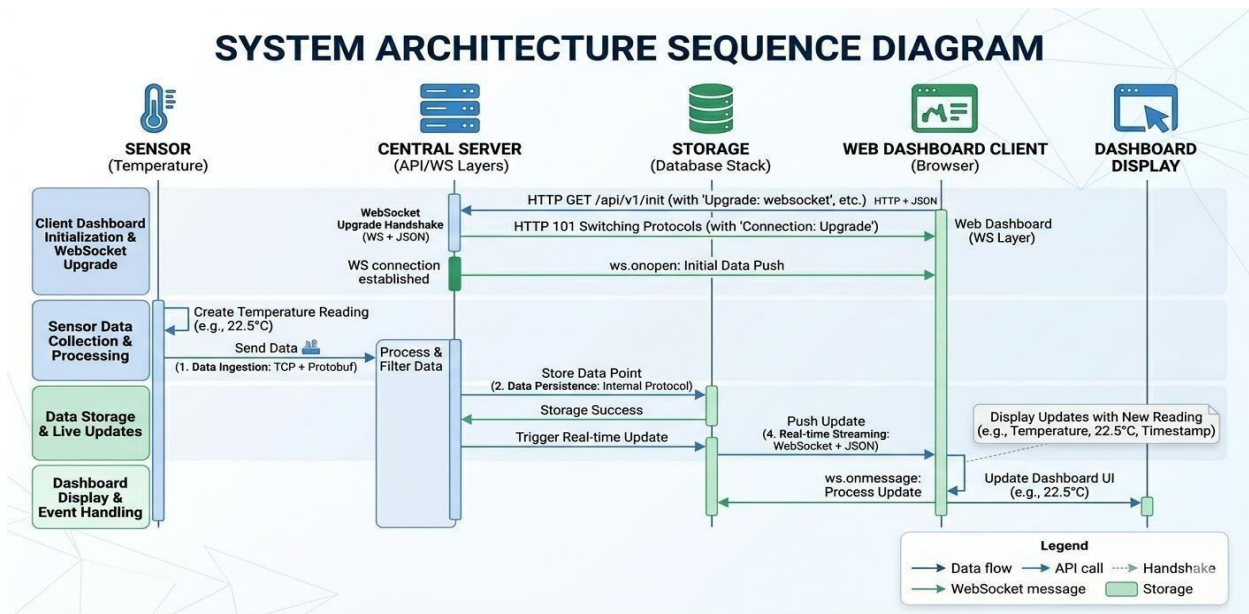


Figure 2: Sequence Diagram (sensor → server → queue → database → WebSocket broadcast → dashboard render, including WebSocket upgrade handshake)

(C). DATA MODEL

The system maintains two core persistent entities: Sensor representing registered or auto-discovered measurement devices, and Reading representing individual telemetry data points.

The Sensor entity includes fields for a unique identifier (string ID), human-readable name, measurement unit (e.g., "celsius"), optional location string, registration timestamp, and an active status flag.

The Reading entity contains a UUID primary key, a foreign key reference to its sensor, a double-precision floating point value, and a timestamp indicating when the measurement was taken.

For efficient querying, indexes are created on the combination of sensor ID and timestamp (descending) as well as timestamp alone.

The complete Protocol Buffers schema is defined in a .proto file and includes a service definition with RPC methods for publishing a single reading (PublishReading) and streaming historical or real-time readings (StreamReadings).

This protobuf definition serves as the contract for potential gRPC extensions, though the current implementation uses JSON over HTTP.

Below is the full Protocol Buffers schema file (proto/telemetry.proto):

```

<!-- proto
syntax = "proto3";
package telemetry;

import "google/protobuf/timestamp.proto";

service TelemetryService {
  rpc PublishReading (PublishRequest) returns (PublishResponse);
  rpc StreamReadings (StreamRequest) returns (stream Reading);
}

message Sensor {
  string id = 1;
  string name = 2;
  string unit = 3;
  string location = 4;
  google.protobuf.Timestamp registered_at = 5;
  bool active = 6;
}

message Reading {
  string id = 1;
  string sensor_id = 2;
  double value = 3;
  google.protobuf.Timestamp timestamp = 4;
}

message PublishRequest {
  string sensor_id = 1;
  double value = 2;
}

message PublishResponse {
  bool accepted = 1;
}

```

The corresponding SQL schema for PostgreSQL uses TEXT primary keys for sensors (allowing humanreadable IDs like "warehouse-temp-03"), UUID for reading records generated by the database, and TIMESTAMPTZ for all timestamps to maintain timezone-aware data.

Foreign key constraints with ON DELETE CASCADE ensure referential integrity when sensors are decommissioned

(D). REST ENDPOINT SPECIFICATION

The server exposes four REST endpoints used for data ingestion, sensor management, and historical data retrieval.

Each endpoint is defined by its path, HTTP method, request and response formats, expected status codes, and a concrete usage example.

The primary ingestion endpoint, POST /api/v1/readings, accepts a JSON payload containing sensor_id and value, and returns an HTTP 201 Created response along with the generated reading UUID.

The GET /api/v1/sensors endpoint is used to retrieve registered sensors and supports optional query parameters for filtering active sensors and limiting the number of results returned.

The GET /api/v1/readings endpoint provides paginated access to historical sensor data, allowing filtering by sensor ID, time range, and offset/limit parameters for efficient data retrieval.

In addition, the WebSocket endpoint /ws/live operates over a different communication protocol, where clients first send a subscription message during connection setup, after which the server continuously streams live sensor readings to all subscribed clients in real time.

Table 1: REST Endpoint Specifications

Path	Method	Request Body	Response Body	Status Codes	Example
/api/v1/readings	POST	{"sensor_id": "string", "value": double}	{"id": "uuid", "status": "accepted"}	201, 400, 404, 500	POST /api/v1/readings → { "sensor_id": "temp-01", "value": 23.5 }
/api/v1/sensors	GET	(query params: ?active=true&limit=100)	{"sensors": [{"id", "name", "unit", "location", "active"}]}	200, 500	GET /api/v1/sensors? active=true
/api/v1/readings	GET	(query: ?sensor_id=string& limit=50&offset=0& from=...&to=...)	{"readings": [...], "total": 123}	200, 400, 500	GET /api/v1/readings? sensor_id=temp-01& limit=10
/ws/live	WebSocket	{"type": "subscribe", "sensor_ids": ["temp-01"]}	{"type": "reading", "data": {...}}	101 (upgrade), 400, 500	WebSocket handshake then subscription message

(E). YAML CONFIGURATION SCHEMA

Sensor devices are configured using a YAML file that defines the sensor identity, server connection parameters, and reading behavior.

The configuration schema includes top-level sections for sensor details (such as ID, name, unit, and optional location), server settings (including URL, API key, timeout, and retry policy), and reading behavior (such as interval, jitter, and batch size).

Each field has a clearly defined type to ensure consistency, validation, and predictable parsing of configuration data at runtime.

The API key field supports environment variable substitution using the `${ENV_VAR}` syntax, which improves security by avoiding hardcoding sensitive credentials and allows flexible deployment across different environments without modifying the configuration file.

The reading behavior section controls how frequently sensors generate data and introduces optional jitter to simulate real-world sensor variability and reduce synchronized network spikes.

The retry policy is implemented using exponential backoff, where each failed connection attempt increases the waiting time exponentially until the maximum number of retry attempts is reached.

This approach prevents overwhelming the server during outages or network instability by spacing out repeated connection attempts.

Once the retry limit is exceeded, the system either stops further retries or triggers a failure state depending on the server configuration and deployment requirements.

This design improves system resilience, ensures controlled recovery behavior, and prevents continuous request flooding during persistent connection failures.

Table 2: YAML Configuration Schema Structure

Field	Type	Required	Description
• SENSOR			
sensor.id	string	Yes	Unique identifier for this sensor
sensor.name	string	Yes	Human-readable display name
sensor.unit	string	Yes	Measurement unit (e.g. °C, %, lux)
sensor.location	string	No	Free text location label
• SERVER			
server.url	string	Yes	API endpoint URL
server.api_key	string	No	Optional authentication key
server.timeout_seconds	integer	No	Request timeout in seconds Default: 5
• SERVER.RETRY			
server.retry.max_attempts	integer	No	Maximum number of retry attempts Default: 3
server.retry.backoff_factor	float	No	Exponential backoff multiplier Default: 1.0
• READING			
reading.interval_seconds	integer	Yes	Pause between readings (seconds)
reading.jitter_seconds	integer	No	Random jitter added to interval
reading.batch_size	integer	No	Readings per batch submission

Legend:
 Required — field must be present for the system to start.
 Optional — field may be omitted; defaults apply where shown.

```

yaml
sensor:
  id: "warehouse-temp-03"
  name: "North Warehouse Temperature"
  unit: "°C"
  location: "Rack D7"

server:
  url: "https://dashboard.mycompany.com"
  api_key: "${SENSOR_API_KEY}"
  timeout_seconds: 5
  retry:
    max_attempts: 5
    backoff_factor: 2.0

reading:
  interval_seconds: 15
  jitter_seconds: 2
  batch_size: 1

```

(F). CONCURRENCY MODEL

The server is implemented entirely using AsyncIO with a single event loop and no operating system threads, except for optional fallback cases involving blocking database drivers.

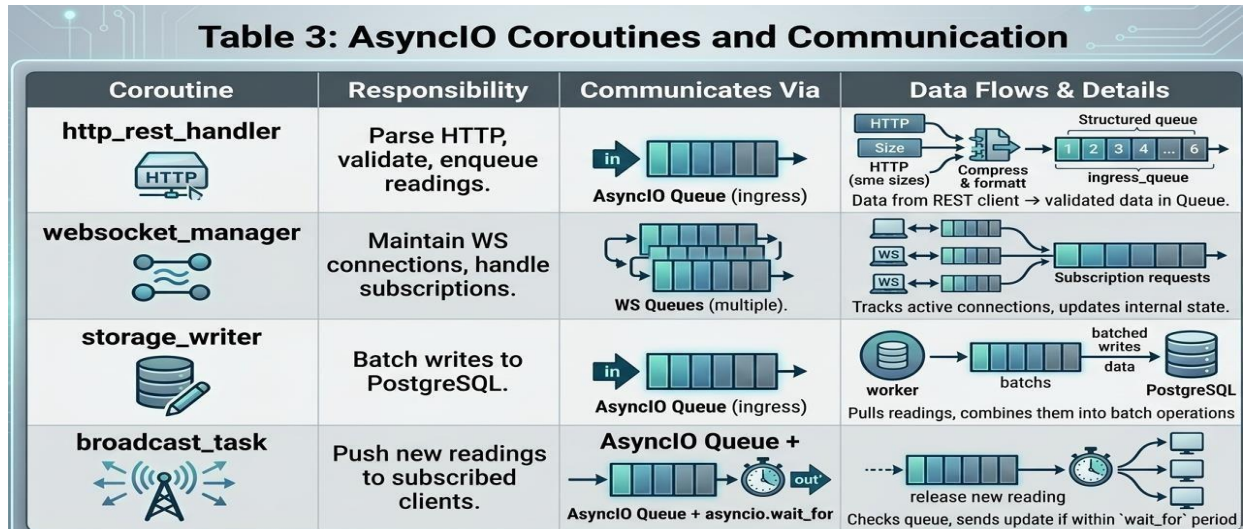
The system runs four main coroutines concurrently within the event loop. The `http_rest_handler` coroutine processes incoming HTTP requests, validates sensor readings, and places valid data into a shared AsyncIO queue for further processing.

The `websocket_manager` coroutine manages all active WebSocket connections, assigns each client a bounded outgoing queue, and handles client subscriptions for real-time updates.

The `storage_writer` coroutine continuously consumes readings from the ingress queue and performs batched insert operations into the database using an asynchronous driver such as `asyncpg`.

The `broadcast_task` coroutine listens for new readings either from a database notification channel or an internal queue and forwards them to all subscribed WebSocket clients.

These coroutines communicate primarily through AsyncIO queues and shared in-memory structures, relying on the single-threaded event loop to avoid race conditions and eliminate the need for explicit locking mechanisms.



When a WebSocket client (consumer) is too slow to keep up with the incoming message stream, the system uses a controlled message-dropping and backpressure strategy.

Each WebSocket connection is assigned a bounded outgoing queue with a maximum capacity of 100 messages.

During broadcasting, if a client's queue is full, the `queue.put_nowait()` operation triggers an `asyncio.QueueFull` exception.

In this case, the server logs a warning indicating a slow consumer and drops the message for that client without retrying, while continuing to send updates to other clients.

If a client's queue remains full continuously for more than 10 seconds, the server treats the client as unhealthy and forcibly closes the WebSocket connection using error code 1008 (policy violation).

The client is then required to reconnect and retrieve missed data through the REST API.

On the ingestion side, sensor-to-server communication uses a bounded queue of 10,000 readings.

If the database becomes slow and the queue reaches capacity, the `await queue.put()` call blocks the HTTP handling coroutine.

This creates natural backpressure, causing sensor requests to be delayed and eventually time out, prompting sensor-side retries.

This mechanism ensures controlled load propagation through the system and prevents uncontrolled memory growth or system overload.

(G). FAILURE HANDLING

The system implements distinct failure handling strategies for three critical scenarios.

When a sensor disconnects (due to network interruption or process crash), the sensor-side client library detects the failure via HTTP timeout or connection reset and automatically initiates an exponential backoff retry sequence as defined in the YAML configuration (default: up to 5 attempts with backoff factor 2.0).

The server retains no in-memory state for disconnected sensors, and any readings that were in transit during the disconnection are lost unless the sensor implements local persistence and replay.

Duplicate readings are handled idempotently if the sensor includes an idempotency key header, though the base system does not require this.

When the storage layer fails (PostgreSQL becomes unreachable or returns errors like disk full with error code 53100), the storage writer coroutine catches the asyncpg connection error or operational error, logs a CRITICAL message, and enters a failure loop where it stops consuming from the ingress queue, allowing the queue to back up.

The health check endpoint returns HTTP 503 Service Unavailable once the queue fills beyond 90% capacity. When the database recovers, the storage writer automatically reconnects and resumes consumption without data loss, provided the ingress queue did not overflow its 10,000-item limit—if overflow occurs, the oldest readings are dropped (first-in, first-out eviction), and an alert is logged.

When the server is overloaded (defined as the ingress queue fill rate exceeding the drain rate for 30 consecutive seconds), the server begins rejecting new POST requests with HTTP 429 Too Many Requests and includes a Retry-After: 5 header instructing sensors to wait 5 seconds before retrying.

Concurrently, the WebSocket manager reduces broadcast frequency for non-critical messages (e.g., status updates) and may disconnect the least active WebSocket clients to preserve resources for ingestion.

Monitoring metrics including queue depth, request latency percentiles, and database connection pool usage are exported via a /metrics endpoint for Prometheus integration, enabling auto-scaling decisions in orchestrated deployments.

The system also implements graceful shutdown on SIGTERM: it stops accepting new connections, waits up to 30 seconds for in-flight requests to complete, drains the ingress queue to a disk-based spill log, closes all WebSocket connections with close code 1001 (going away), and finally exits.

REFERENCES:

- Python Software Foundation. (2026). *asyncio — Asynchronous I/O, event loop, coroutines and tasks*. <https://docs.python.org/3/library/asyncio.html>
- Google. (2026). *Protocol Buffers documentation*. <https://protobuf.dev/>
- Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* (Doctoral dissertation, University of California, Irvine). <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>
- Mozilla Developer Network. (2026). *WebSockets API*. https://developer.mozilla.org/enUS/docs/Web/API/WebSockets_API
- Red Hat. (2026). *What is REST?*. <https://www.redhat.com/en/topics/api/what-is-a-rest-api>
- YAML.org. (2026). *YAML Ain't Markup Language (YAML) specification*. <https://yaml.org/spec/>
- SQLite Consortium. (2026). *SQLite documentation*. <https://www.sqlite.org/docs.html>
- PostgreSQL Global Development Group. (2026). *PostgreSQL documentation*. <https://www.postgresql.org/docs/>